

Leitura e preparação inicial de dados com Python

Análise de Dados

Aléssio Tony Cavalcanti de Almeida

Centro de Ciências Sociais Aplicadas
Universidade Federal da Paraíba
alessio@lema.ufpb.br



CIÊNCIA DE
DADOS PARA
NEGÓCIOS

João Pessoa, 2026

Plano de Aula

Objetivo: compreender o ciclo inicial de análise de dados com Python, desde a identificação das fontes e formatos até a importação, inspeção e preparação básica de tabelas com pandas.

Competências esperadas ao final da aula:

- reconhecer diferentes fontes e formatos de dados usados em pesquisas aplicadas;
- importar arquivos CSV, TXT de largura fixa e XLSX no Python;
- diagnosticar tipos, dimensões, valores ausentes e problemas básicos de leitura;
- construir pequenos fluxos reprodutíveis de tratamento e análise exploratória.

Leituras de apoio:

- Carvalho et al. (2024): cap. 4 Amaral (2016) Fontelles et al. (2009)
- Provost e Fawcett (2016) Sheppard (2013) Vanderplas (2017)
- Jake (2019) Idris (2014)

Estrutura da Aula

- 1 Introdução
- 2 Fontes de dados
 - Principais fontes de Dados Secundários
 - IBGE
 - Datasus
 - Outras fontes
- 3 Pandas
 - CSV coluna fixa
 - XLS/XLSX
- 4 Atividade Prática

Introdução

Introdução

❓ Como transformar **dados brutos** em **evidências úteis** para tomada de decisão?

❓ Quais requisitos técnicos tornam uma análise **confiável, reprodutível e escalável**?

Ideia central da aula

O valor analítico não está apenas no volume de dados, mas na capacidade de organizar, documentar, importar, validar e transformar esses dados em indicadores consistentes.

Do dado à decisão: pipeline analítico

- ❶ **Problema:** qual decisão precisa ser apoiada?
- ❷ **Fonte:** quais dados representam o fenômeno?
- ❸ **Importação:** como ler os dados corretamente?
- ❹ **Validação:** os tipos, códigos e valores fazem sentido?
- ❺ **Transformação:** como construir indicadores e agregações?
- ❻ **Evidência:** que padrão empírico sustenta a recomendação?

Fontes de dados

Dados, informação e evidência

Dados

- Constituem a matéria-prima da informação
- Dados sem qualidade levam a informações e decisões da mesma natureza

Informação

- Informação é um conjunto de dados organizados com sentido e utilidade para determinado usuário.

Evidência

Informação analisada com método, contexto e critérios de validade, capaz de sustentar uma decisão ou recomendação.

Microdados

- Microdados consistem no menor nível de observação possível de um dado, possibilitando diferentes cálculos estatísticos e avaliações

Natureza

- **Dados primários:** são dados originais (customizados), coletados pelo pesquisador a partir de um dado instrumento
- **Dados secundários:** já foram coletados (usualmente por órgãos governamentais) e estão disponíveis ao público

Estrutura da base

- **Tabela:** estrutura interna de um banco de dados em linhas e colunas
- **Linha:** Cada indivíduo corresponde a uma linha.
- **Coluna:** Cada atributo (indicador) corresponde a uma coluna.

As tabelas relacionam-se umas às outras por meio de **chaves**: por exemplo, ID do indivíduo (**chave primária**) e ID do domicílio (**chave estrangeira**).

Boa prática: dados tabulares organizados

Cada linha deve representar uma unidade de observação; cada coluna, uma variável; cada célula, um único valor.

O que é uma base analítica organizada?

Regra central

Uma base está em formato analítico quando:

- cada **linha** representa uma unidade de observação;
- cada **coluna** representa uma variável;
- cada **célula** contém um único valor;
- cada tabela representa um único tipo de unidade observacional.

Exemplo

Em uma base de estudantes:

- linha: estudante;
- colunas: idade, curso, CRA, situação acadêmica;
- célula: valor observado para uma variável em um estudante.

Extensões mais conhecidas

- **Arquivo de texto** – .txt .csv:
 - ① delimitado por algum caractere: “,” ou “;” (ou qualquer outro)
 - ② delimitado por espaço em branco ou por uma tabulação
 - ③ coluna fixa com dicionário
- **Excel** – .xls .xlsx
- **Stata/SPSS** – .dta .sav
- **R** – .RData .rds
- **Outros** –
.xml .json .html .dbc .parquet .feather .shp SQL MongoDB API
- **Importante:** atenção com arquivos com strings devido ao **encoding do texto** – mais usados são LATIN-1 (ISO-8859-1) e UTF-8 – para evitar problemas com acentuações

Dados secundários: IBGE

- Uma das fontes de dados com maior diversificação disponível no país para pesquisa acadêmica e técnica
- Dispõe de informações sobre população, municípios, geociências (informações espaciais), economia (PIB, setores, empresas), inovação, consumo, mercado de trabalho, suplementos de saúde etc.
- A maior parte dos dados encontram-se disponíveis gratuitamente para download. Alguns dados e outras publicações podem ser adquiridos em <loja.ibge.gov.br>
- Em termos operacionais, os microdados providos pelo IBGE, em geral, requerem dicionários – gerando um trabalho adicional para sua leitura

Dados secundários: IBGE

Censo Demográfico

- Periodicidade decenal
- Os microdados são disponíveis para os resultados gerais da amostra
- Representatividade para todas as unidades municipais do país (setores censitários)
- Informações em nível do indivíduo e do domicílio
- Variáveis demográficas e socioeconômicas
- Complexidade de manipulação e tratamento dos dados: **Intermediário-Alto**

Dados secundários: IBGE

PNAD / PNAD Contínua

- A PNAD tradicional foi substituída pela PNAD Contínua; os resultados atuais são divulgados em recortes mensais, trimestrais e anuais, conforme o indicador
- Representatividade para todas as unidades federativas do país
- Informações em nível do indivíduo e do domicílio
- Variáveis demográficas e socioeconômicas
- Em algumas edições possui suplementos especiais: saúde, justiça, tabagismo...
- **Desenho amostral complexo**
- Complexidade de manipulação e tratamento dos dados: **Intermediário-Baixo**

Dados secundários: IBGE

PNS

- Pesquisa domiciliar de saúde com edições nacionais recentes em 2013 e 2019; antes de usar, verificar a edição e o desenho amostral no portal do IBGE
- Representatividade para todas as unidades federativas do país
- Informações em nível do indivíduo e do domicílio
- Variáveis demográficas, socioeconômicas e de **saúde** (hábitos, consumo de medicamentos, indicadores antropométricos, hipertensos, diabéticos...)
- **Desenho amostral complexo**
- Complexidade de manipulação e tratamento dos dados: **Intermediário-Baixo**

Dados secundários: IBGE

POF

- Realização a cada seis anos
- Representatividade para todas as unidades federativas do país
- Informações em nível do indivíduo e do domicílio
- Variáveis demográficas, socioeconômicas e de consumo (alimentar, remédios, lazer etc.)
- Em 2008/2009, contou com o Inquérito Nacional de Consumo Alimentar
- **Desenho amostral complexo**
- Complexidade de manipulação e tratamento dos dados: **Avançado**

Datasus

- Vigitel: anual (2006-atual), representatividade espacial (capitais)
- SIA: mensal, representatividade espacial (municípios)
- SIH: mensal, representatividade espacial (municípios)
- SIM: anual, representatividade espacial (municípios)
- SINASC: anual, representatividade espacial (municípios)
- SINAN: anual, representatividade espacial (municípios)
- CNES: mensal, representatividade espacial (bairros)
- Em geral, os dados dos Sistemas de Informações encontram-se na extensão .dbc!
Anteriormente, precisaríamos do **Tabwin**; com Python, é possível automatizar a leitura por meio de pacotes como `pysus` ou `pyreaddbc`

Fontes diversas

- INEP: ANA, Censo Escolar, Censo Superior, SAEB, Enade, Enem, Enceja
- MTE: Rais, Caged
- CNPq: Lattes, DGP, Bolsas, Projetos
- Portal da Transparência
- Dados Abertos: MEC, CAPES, Agências (ANP, ANVISA...)
- TSE: dados eleitorais
- STN: Siconfi, Finbra
- SAGRES: licitações, empenhos, liquidações
- MDS: CadÚnico, Censo SUAS
- Yahoo!, Google, X, MapBiomas...

Fontes públicas: o que muda na prática?

Fonte	Formato comum	Desafio operacional
IBGE	TXT, CSV, XLSX	dicionários, pesos amostrais, largura fixa
INEP	CSV, XLSX	muitas colunas, códigos educacionais, granularidade
DATASUS	CSV, DBC, TABNET	formatos próprios, filtros, séries históricas
Portais abertos	CSV, JSON, API	padronização, atualização e documentação
Bancos de dados	SQL	conexão, permissões e modelagem relacional

Mensagem principal

A fonte define o tipo de cuidado necessário na importação, validação e documentação da análise.

Pandas

O que é o Pandas?

- Biblioteca Python para manipulação e análise de dados
- Estruturas de dados principais: Series e DataFrames.

Por que usar o Pandas?

- Facilidade de uso e alta performance.
- Integração com outras bibliotecas (NumPy, Matplotlib, etc.).

Documentação oficial: <<https://pandas.pydata.org/docs/>>

Fluxo mínimo de análise com Pandas

- ❶ **Ler:** carregar os dados com os parâmetros corretos.
- ❷ **Inspecionar:** verificar dimensões, tipos, colunas e primeiras linhas.
- ❸ **Validar:** identificar valores ausentes, duplicidades, códigos inválidos e problemas de escala.
- ❹ **Transformar:** selecionar variáveis, filtrar casos, criar indicadores e agregar informações.
- ❺ **Comunicar:** produzir tabelas, gráficos e sínteses interpretáveis.

Primeiros passos com Pandas

Inicializando o Pandas

```
pip install pandas
```

```
import pandas as pd
```

Criando Séries

```
data = [1, 2, 3, 4, 5]  
index = ["a", "b", "c", "d", "e"]  
series = pd.Series(data, index=index)
```

Criando um DataFrame

```
data = {"col1": [1, 2, 3], "col2": [4, 5, 6]}  
df = pd.DataFrame(data)
```


Atributos dos dados no Pandas

Tipos de objetos ou tipagem de colunas

Cada coluna de um pandas DataFrame receberá uma tipagem especial.

Python	Pandas	Descrição
string	object	Texto (strings variadas)
int	int64	Número inteiro
float	float64	Número com casas decimais
bool	bool	Valores booleanos (True/False)
datetime	datetime64	Data e/ou hora

Kit mínimo para diagnosticar uma base

```
df.shape # dimensoes da base

df.head() # primeiras linhas

df.info() # tipos das colunas e valores nao nulos

df.columns # nomes das colunas

df.isna().sum() # valores ausentes por coluna

df.duplicated().sum() # duplicidades

df.describe(include="all") # estatisticas descritivas
```

Regra prática

Nunca comece uma análise por gráficos ou modelos. Comece entendendo a estrutura, os tipos, os valores ausentes e a consistência da base.

Inspeção básica de um objeto DataFrame

Podemos usar vários métodos para obter informações sobre um DataFrame:

- **.dtypes** – tipagem das colunas
- **.columns** – nome das colunas
- **.shape** – formato do DataFrame
- **.info()** – sumário de informações do DataFrame

Explorando algumas linhas da planilha de dados no pandas

- **.head(n)**: Retorna as primeiras 'n' linhas (default é 5).

```
print(my_dataframe.head(2))
```

- **.tail(n)**: Retorna as últimas 'n' linhas (default é 5).

```
print(my_dataframe.tail(1))
```

- **.loc[]**: Seleção baseada em rótulos (nomes de índice e colunas).

```
# Seleciona a linha com indice A e a coluna Produto da Series  
# Supondo que my_series tenha indice A  
print(my_dataframe.loc[0, 'Produto'])
```

- **.iloc[]**: Seleção baseada em posição inteira (índices numéricos de linha e coluna, começando do 0).

```
# Seleciona a primeira linha e a segunda coluna  
print(my_dataframe.iloc[0, 1])
```

Funções básicas: verbetes principais

Verbos	Função
<code>filter(items=[])</code>	seleciona um subconjunto de colunas
<code>query()</code>	seleciona um subconjunto de dados (linhas) por condições lógicas
<code>rename()</code>	renomeia colunas ou índices de interesse
<code>sort_values()</code>	reordena linhas com base nos valores de uma ou mais colunas
<code>assign()</code>	cria novas colunas, mantendo as existentes
<code>groupby()</code>	agrupa dados com base em uma ou mais colunas para aplicar funções agregadas
<code>agg()</code>	condensa múltiplos valores em um único, redução de dimensão (usado com <code>groupby</code>)
<code>sample()</code>	gera amostra aleatória de linhas do DataFrame
<code>merge()</code>	realiza junções de tabelas (left, right, inner, outer)
<code>reset_index()</code>	reinicia a indexação da tabela de dados, transformando o índice atual em coluna

Problemas comuns ao importar dados

Sintoma	Causa provável	Correção
Uma única coluna gigante Acentos quebrados	Separador incorreto Encoding incompatível	Ajustar sep Testar <code>encoding="utf-8"</code> ou <code>"latin1"</code>
Números viram texto	Decimal ou milhar incompatível	Ajustar decimal e thousands
Colunas com nomes errados Datas como texto	Cabeçalho em linha diferente Falta de conversão temporal	Usar <code>skiprows</code> ou <code>header</code> Usar <code>parse_dates</code> ou <code>pd.to_datetime()</code>
Valores ausentes não reconhecidos	Códigos próprios da base	Usar <code>na_values</code>

Importação de Dados

Importando Dados CSV/TXT delimitados por caractere (,;| etc)

Usando a função `read_csv` do pacote **pandas**

Sintaxe Básica:

```
# Ajuda
import pandas as pd
help(pd.read_csv)

# Importando dados
dados = pd.read_csv("<nome do arquivo>", sep = "<separador>")
```

Importação de Dados

Importando Dados CSV/TXT delimitados por caractere (,;| etc)

Principais parâmetros da função:

sep	caractere delimitador dos dados
usecols	lista de variáveis a serem carregadas
decimal	caractere separador da casa decimal
nrows	número de linhas do arquivo para ler
skiprows	número de linhas a serem puladas
na_values	string para reconhecer NA/NaN
encoding	encoding de texto

Importação de Dados

Importando Dados CSV/TXT delimitados por caractere (,;| etc)

EXEMPLO: DADOS HISTÓRICOS DO IBOVESPA

Arquivo de dados no repositório do Curso

Fonte: Yahoo Finance.

```
# Copie o arquivo CSV para seu diretorio de trabalho
# Importando dados
dados = pd.read_csv("ibovespa_1998a2016.csv", sep=",", decimal=".")

# Visualizar primeiras linhas
dados.head()
```

Importação de Dados

Importando dados TXT fixos por colunas

read_fwf do pacote **pandas**. **Sintaxe Básica:**

```
# Ajuda
help(pd.read_fwf)
# Importando dados
dados = pd.read_fwf("<arquivo>", colspecs = [posicao], names = [nomes]
                )
```

Importação de Dados

Importando dados TXT fixos por colunas

EXEMPLO: PESQUISA NACIONAL DE AMOSTRA POR DOMICÍLIO (PNAD):

Baixe o arquivo de dados no repositório do Curso.

Fonte: IBGE.

Abrir o **dicionário de variáveis de pessoas** no Excel/LibreOffice.

Vamos importar as seguintes variáveis na **sequência de posições**:

- Ano (posição 1/tamanho 4)
- Unidade da Federação (posição 5/tamanho 2)
- Sexo (posição 18 / tamanho 1)
- Idade (posição 27/ tamanho 3)
- Cor (posição 33 / tamanho 1)

Importação de Dados

Como a leitura de dados de colunas fixas requer um dicionário com a posição das colunas, no **Pandas** será preciso ter atenção para a indexação de posição do **Python** que inicia no zero.

Variável	IBGE			Pandas	
	Início (I)	Tamanho (T)	Final $I + T - 1$	Início $I - 1$	Final $(I-1) + T$
Ano	1	4	4	0	4
UF	5	2	6	4	6
Sexo	18	1	18	17	18
Idade	27	3	29	26	29
Cor	33	1	33	32	33
Renda					

a) Importação de Dados (Tamanho da coluna)

Importando dados TXT fixos por colunas

Lógica das entradas de tamanho de variáveis (colunas fixas):

```
# Criamos uma lista de nomes para os campos importados
nomes = ['ano', 'uf', 'sexo', 'idade', 'cor']

# Posicao inicial e final indexadas para o pandas
posicao = [(0, 4), (4, 6), (17, 18), (26, 29), (32, 33)]

# Leitura do arquivo de dados passando as posicoes e nomes
df = pd.read_fwf('pnad_anual_2014/PES2014.txt',
                 colspecs = posicao, names = nomes)
# Ler primeiras linhas
df.head()
```

Importação de Dados: XLS/XLSX

Função principal: `pd.read_excel()`

Parâmetro	Uso principal
<code>sheet_name</code>	escolhe a aba da planilha
<code>usecols</code>	seleciona colunas específicas
<code>skiprows</code>	ignora linhas iniciais não pertencentes à base
<code>nrows</code>	limita a quantidade de linhas lidas
<code>header</code>	define a linha que contém os nomes das colunas
<code>names</code>	atribui nomes manualmente às colunas
<code>na_values</code>	define códigos que devem ser tratados como ausentes

```
dados = pd.read_excel(  
    "arquivo.xlsx",  
    sheet_name="Planilha1",  
    usecols=["ano", "uf", "valor"],  
    na_values=["-", "NA", ""]  
)
```

Importação de Dados

Importando Dados XLS/XLSX

EXEMPLO: DIVISÃO TERRITORIAL DO BRASIL (DTB)

Fonte: IBGE.

```
# Importando dados
dados = pd.read_excel("dtb_uf.xlsx")
# Visualizar primeiras linhas
dados.head()
```

Importação de dados

Importando dados XLS/XLSX

EXEMPLO: ÍNDICE DE DESENVOLVIMENTO DA EDUCAÇÃO BÁSICA

Fonte: INEP

```
# Importando dados
df = pd.read_excel("ideb5_2015.xlsx", skiprows=8, header=None)
df.head()
```


Checklist de validação após importar uma base

```
# Dimensao da base
print(df.shape)

# Tipos e valores ausentes
df.info()
df.isna().mean().sort_values(ascending=False)

# Duplicidades em uma chave esperada
df.duplicated(subset=["id_municipio", "ano"]).sum()

# Estatísticas rápidas das variáveis numéricas
df.describe().T
```

Exemplo de encadeamento reprodutível

```
resultado = (  
    df  
    .query("uf == 'PB'")  
    .assign(  
        gasto_educ_pc = lambda x: x["gasto_educacao"] / x["populacao"]  
        ,  
        pct_educacao = lambda x: 100 * x["gasto_educacao"] / x["  
            gasto_total"]  
    )  
    .groupby("regiao_imediata", as_index=False)  
    .agg(  
        gasto_educ_pc_medio=("gasto_educ_pc", "mean"),  
        pct_educacao_medio=("pct_educacao", "mean"),  
        n_municipios=("id_municipio", "nunique")  
    )  
)
```

Atividade Prática

Atividade prática 0: diagnóstico inicial de uma base

Contexto: usar uma base pública para produzir um diagnóstico preliminar. Antes de qualquer análise substantiva, é necessário verificar se os dados foram importados corretamente e se estão minimamente consistentes.

- 1 importar a base usando os parâmetros adequados;
- 2 identificar número de linhas, colunas e tipos de variáveis;
- 3 verificar valores ausentes e duplicidades;
- 4 identificar pelo menos três problemas potenciais de qualidade;
- 5 produzir uma tabela-resumo com indicadores básicos;
- 6 escrever uma conclusão curta sobre a confiabilidade inicial da base.

Entrega

Notebook com código executável, comentários explicativos e uma síntese final de até 10 linhas.

Critérios de avaliação da atividade

Critério	O que será observado
Importação correta	separador, encoding, decimal, cabeçalho e nomes de colunas
Inspeção da estrutura	uso adequado de <code>shape</code> , <code>info</code> , <code>head</code> , <code>dtypes</code>
Diagnóstico de qualidade	ausentes, duplicados, inconsistências e valores improváveis
Transformação mínima	criação de indicadores ou agregações simples
Comunicação	clareza da conclusão e reprodutibilidade do notebook

Atividade prática 1: gasto educacional e resultado escolar

Dados do Finbra: finbra_pb_2015.csv

- 1 Filtrar dados para os municípios da Paraíba
- 2 Obter dados dos gastos em educação e os gastos totais
- 3 Calcular o percentual dos gastos em educação
- 4 Cruzar informações com divisão territorial (dtb_municipios.csv) e desenvolver análises por níveis regionais do Estado.

Dados do Inep: tdi_municipios_2015.xlsx

- 1 Filtrar dados para os municípios da Paraíba
- 2 Obter a TDI para o ensino fundamental
- 3 Cruzar as informações com os gastos em educação e fazer uma análise de correlação.

Atividade prática 2: tratamento e agregação do IDEB

- 1 Ler dados `ideb5_2015.xlsx`
- 2 Consultar o nome das variáveis após a leitura
- 3 Selecionar as seguintes variáveis: UF, MUN (COD e NOME), COD_ESCOLA (COD e NOME), REDE, APROVA_1A5 e IDEB (2005 a 2015)
- 4 Transformar variáveis lidas de forma errada em numéricas
- 5 Criar uma nova tabela de dados **med_ideb**: calculando a média do IDEB anual por município (Existe algum viés nesse cálculo?)
- 6 Criar uma nova tabela de dados **ideb_est**: calculando a média e o desvio-padrão do IDEB anual por município (Existe algum viés nesse cálculo?)
- 7 Contar o total de escolas por estado e rede de ensino

Atividade prática 3: permanência no Ensino Superior Federal I

Atenção metodológica

O indicador proposto é uma proxy agregada e não mede evasão individual. A interpretação deve destacar limitações de coorte, migração entre cursos, retenção e tempo heterogêneo de integralização.

- 1 Ler os **microdados do Censo Superior** para os anos de 2016 e 2022 (subconjunto de colunas de interesse para a análise da evasão). Garantir se os dados numéricos foram lidos corretamente.
- 2 Transformar variáveis lidas de forma errada em numéricas, filtrar apenas rede federal de ensino
- 3 Criar uma proxy de não conclusão esperada por curso e universidade, reconhecendo explicitamente suas limitações: $EV_t = 1 - \frac{QT_CONC_t}{QT_ING_{t-7}}$

Atividade prática 3: permanência no Ensino Superior Federal II

- 4 Identificar na tabela de dados com o índice de evasão o nome (sigla) das universidades.
- 5 Identificar as universidades com os maiores indicadores de evasão (top 10).
- 6 Realizar uma distribuição de frequência (histograma empírico) da evasão entre as universidades.
- 7 Identificar os cursos com os maiores indicadores de evasão (top 10).
- 8 Identificar as áreas de curso com os maiores indicadores de evasão (top 5).
- 9 Fazer uma síntese estatística da evasão entre as universidades.
- 10 Fazer uma síntese estatística da evasão pelas macrorregiões brasileiras: max, min, média, mediana, q10, q90, desvio-padrão.
- 11 Salvar as tabelas de interesse no formato EXCEL.

Referências I

 AMARAL, F. *Introdução à Ciência de Dados: mineração de dados e big data*. Rio de Janeiro: Alta Books, 2016.

 CARVALHO, A.; MENEZES, A.; BONIDIA, R. *Ciência de Dados: Fundamentos e Aplicações*. Rio de Janeiro: LTC, 2024. 1–354 p.

 FONTELLES, M. J. et al. Metodologia da pesquisa científica: diretrizes para a elaboração de um protocolo de pesquisa. *Revista Paraense de Medicina*, v. 23, n. 3, p. 1–8, 2009.

 IDRIS, I. *Python Data Analysis*. [S.l.]: Packt Publishing, 2014. ISBN 9781783553358.

 JAKE, V. *Python Data Science Handbook*. [S.l.: s.n.], 2019. v. 53. 1689–1699 p. ISSN 1098-6596. ISBN 9788578110796.

 PROVOST, F.; FAWCETT, T. *Data Science para negócios*. Rio de Janeiro: Alta Books, 2016.

Referências II

 SHEPPARD, K. *Introduction to Python for Econometrics, Statistics and Data Analysis*. 2. ed. University of Oxford, 2013. 393 p. Disponível em: <[{\% }5Cnpapers3://publication/uuid/790C53F4-2413-4492-8E88-8D54F5A30>.](http://onlinelibrary.wiley.com/doi/10.1111/j.1435-5957.2010.00279.x/full)

 VANDERPLAS, J. *Python Data Science Handbook: Essential Tools for Working with Data*. [S.l.]: O'Reilly, 2017. ISBN 9781491912058.